Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON OCT, 2025

Clique

Security Assessment

SAFEEDGES.IN

Clique Final Audit Report

Table of Contents

APPROACH & METHODS Raffle

This report has been prepared for **Raffle** to identify potential issues and vulnerabilities within the source code of the Raffle project, as well as any contract dependencies not part of officially recognized libraries.

A comprehensive audit was conducted using a combination of **Formal Verification**, **Manual Review**, and **Static Analysis** techniques.

The auditing process focused on the following key aspects:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase for compliance with current best practices and industry standards.
- Ensuring the contract logic aligns with the projects intended specifications and objectives.
- Cross-referencing contract structure and implementation with similar smart contracts developed by leading industry projects.
- Conducting a thorough line-by-line manual review of the entire codebase by experienced security experts.

The security assessment produced findings ranging from critical to informational severity levels. We strongly recommend addressing these findings to ensure adherence to top-tier security standards and industry best practices.

Additionally, we suggest the following improvements to further strengthen the Raffle projects overall security posture:

- Test the smart contracts against both common and uncommon attack scenarios.
- Improve general coding practices to enhance structure and maintainability.
- Add sufficient unit tests to cover all possible use cases.

Include clear and consistent comments for each function to improve readability—especially for publicly verified contracts.

Ensure transparency regarding privileged operations once the protocol is deployed and live.

Executive Summary

Item	Details
Scope	`CliqueLockHook`, `Distributor`, `BaseDistributor-`, `TransferHook`
Timeframe	September 18–28, 2025
Methodology	Manual code review, static analysis, and best practices evaluation
Severity Summary	CRITICAL
Overall Risk Level	MEDIUM
Current Status	**All Issues Closed**

> *This audit reviewed the core distribution and streaming hook contracts to identify security vulnerabilities and logic flaws. We identified critical, high, medium, low, informational, and gas findings as summarized above. All issues are currently open and require remediation.*

Techniques and Methods

Code quality checks

Best practices & documentation review

Gas efficiency analysis

Reentrancy & ERC compliance checks

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open	Resolved
Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	**Partially Resolved**
Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity	Count	Percentage	Status	Fixed	Commits
CRITICAL	1	8.33%	Fixed	**1 / 1 Fixed (100%)**	836c1c8
HIGH	1	8.33%	Fixed	**1 / 1 Fixed (100%)**	836c1c8
MEDIUM	1	8.33%	Acknowledged	**0 / 1 Fixed (0%)**	836c1c8
LOW	4	33.33%	Partial	**2 / 4 Fixed (50%)**	836c1c8
INFORMA-TIONAL	3	25.00%	Fixed	**3 / 3 Fixed (100%)**	836c1c8
GAS	2	16.67%	Partial	**1 / 2 Fixed (50%)**	836c1c8

Detailed Findings

[C-01] Unchecked overwrite of streamIds mapping in CliqueLockHook.execute() function

Status Fixed

Description execute() function unconditionally sets `streamIds[_root][_recipient] = streamId` after creating a stream. The function performs no access-control check preventing arbitrary callers from invoking execute(), nor does it prevent a second call from overwriting an existing streamId. As a result, an attacker (or an accidental duplicate execution by the distributor) can replace the CliqueLockHook's reference to an earlier stream for the same (root, recipient) pair.

```
function execute(address _vault, address _recipient, bytes32 _root, uint256 _allo-
catedamount, bytes memory _extra)
 external
 returns (uint256 _consumed)
{
....
_distributor().hookTransfer(address(this), _amount);
 token.safeApprove(LOCK, _amount);
 uint256 streamId = CliqueLock(LOCK).createStream(_streamConfig);
@>>     streamIds[_root][_recipient] = streamId;
 return _amount;
}
```

The `claim()` function will only point to the most-recent `streamId`. If overwritten, legitimate, earlier streams remain no longer addressable through the hook.

```
function claim(bytes32 _root, address _recipient) external {
@>>     uint256 streamId = streamIds[_root][_recipient];
 CliqueLock(LOCK).claim(streamId);
}
```

Impact Threat actor can cause low cost state corruption and griefing attack.

Attack Path:

1. The attacker deploys a contract that imports `IDistributor.sol` and implements the minimal functions the hook calls and funds the contract

```
import "./IDistributor.sol";
contract Attacker is IDistributor {
 function configurationId(bytes32 root) external view returns (bytes32) {
 return someConfigurationId; // return a config that maps to a valid StreamPreset
}
 function hookTransfer(address to, uint256 amount) external {
 IERC20(token).transfer(to, amount);
}
// implement other IDistributor functions as needed by the hook
}
```

2. Attacker calls execute on the hook using the victims tuple: `execute(victimVault, victimRecipient, victimRoot, _allocatedAmount, _extra)`.

Here _extra decodes to _amount = 1.

3. The hook approves `LOCK` and calls `CliqueLock.createStream`(). The call succeeds (streamConfig valid) and returns `attackerStreamId`

4. Hook overwrites mapping : `streamIds[victimRoot][victimRecipient] = attackerStreamId;`

5. When the victim `claim(victimRoot, victimRecipient)`, the hook forwards only the attackers streamId to `CliqueLock.claim()`. The victim receives the tiny attacker-funded amount; the original stream is no longer referenced by the hook.

Recommended mitigation Consider enforcing an explicit `onlyDistributor()` modifier to ensure that only the authorized distributor contract can invoke this function.

[H-01] Incorrect Assignment of Immutable `IS_FIXED_START` in `CliqueLockHook.sol`

Status Fixed

Description The contract declares `IS_FIXED_START` as an immutable variable and initializes it to `false`. However, the constructor attempts to assign a new value to `IS_FIXED_START` during deployment. In Solidity, immutable variables cannot be reassigned after declaration; the compiler reserves them for one-time assignment (either at declaration or in the constructor, but not both). This results in a compile-time error, when beign set to `true`.

Impact The contract will fail to compile and cannot be deployed, blocking deployment entirely.

Recommended mitigation Remove the default initialization at the declaration and assign `IS_FIXED_START` only in the constructor, for example:

```
-    bool public immutable IS_FIXED_START = false;
+    bool public immutable IS_FIXED_START;
constructor(address _lock, bool _isFixedStart) {
+        IS_FIXED_START = _isFixedStart;
...
}
```

[M-01] Missing Validation of Total Proportions in `Distributor.setHooks`

Status Acknowledged

Description The `setHooks` function allows the admin to configure multiple strategies for a batch and assigns the `HOOK` role to each strategy's hook contract. While it validates that each individual strategys proportion does not exceed 1e18 (100%), it does not check that the sum of all strategy proportions is `1e18`.

As a result, if the total of all proportions exceeds 1e18, the `_execute` function may attempt to allocate more tokens than the claim `_amount`. This can cause `InsufficientAllocation()` or `ConsumedAmountMismatch()` reverts, effectively preventing legitimate claims from being processed.

Impact Claims can fail unexpectedly, creating a potential DoS scenario for users.

Recommended mitigation Add a validation step in setHooks to ensure the sum of all strategy proportions does not exceed 1e18.

```
function setHooks(bytes32 _configurationId, Strategy[] memory _strategies, uint256 _fall-
backHook)
 external
 onlyRoles(PROJECT_ADMIN)
{
 Strategy[] memory oldStrategies = batchConfigurations[_configurationId].strategies;
 for (uint256 i = 0; i < oldStrategies.length; i++) {
_removeRoles(oldStrategies[i].hook, HOOK);
}
+        uint256 sum = 0;
 for (uint256 i = 0; i < _strategies.length; i++) {
 if (_strategies[i].proportion > 1 ether) {
 revert InvalidStrategies();
}
+            sum += _strategies[i].proportion;
_setRoles(_strategies[i].hook, HOOK);
}
+        if (sum > 1 ether) revert InvalidStrategies();
 if (_fallbackHook > _strategies.length) {
 revert InvalidStrategies();
}
 batchConfigurations[_configurationId].strategies = _strategies;
 batchConfigurations[_configurationId].fallbackHook = _fallbackHook;
 emit HookAdded(_configurationId);
}
```

[L-01] Absence of validation in `BaseDistributor.setActiveWindow`

Status Acknowledged

Description The `setActiveWindow` function allows the admin to configure the start and end timestamps of the airdrops active window. Currently, the function does not validate that the `_endTime` is strictly greater than `_startTime`. This could potentially allow setting an invalid window (e.g., _endTime <= _startTime), which may prevent the airdrop from ever being active.

Impact Misconfigured active window can cause all claims to fail due to the `whenActive` modifier reverting.

Recommended mitigation Add a validation check to ensure `_endTime` > `_startTime`.

```
+   error InvalidActiveWindow()
```

```
function setActiveWindow(uint256 _startTime, uint256 _endTime) external onlyRoles(PRO-
JECT_ADMIN) {
+       if (_endTime <= _startTime) revert InvalidActiveWindow()
 activeWindowStartTime = _startTime;
 activeWindowEndTime = _endTime;
 emit ActiveWindowSet(_startTime, _endTime);
}
```

[L-02] `Distributor.setHooks` allows fallback index equal to `strategies.length`

Status Acknowledged

Description The `setHooks` function allows the admin to configure a fallback hook for a batch by specifying its index in the strategies array. The current validation checks that the fallback index is not greater than strategies.length:

```
if (_fallbackHook > _strategies.length) revert InvalidStrategies();
```

However, this allows `_fallbackHook == strategies.length`, which is out of bounds for the array. In the `_execute` function, the fallback hook is only executed if:

```
if (_batchConfiguration.fallbackHook < _batchConfiguration.strategies.length)
```

Thus, setting _fallbackHook == strategies.length effectively disables the fallback hook. If other hooks do not consume the full `_amount`, totalConsumed != _amount and the claim will revert (`ConsumedAmountMismatch()`), potentially breaking legitimate claims.

Impact Misconfigured fallback index can prevent fallback execution, potentially causing claims to revert even if all other hooks execute correctly.

Recommended mitigation Consider implementing following changes:

```
-       if (_fallbackHook > _strategies.length) {
+       if (_fallbackHook >= _strategies.length) {
revert InvalidStrategies();
```

```
}
```

[L-03] Token Approval Not Reset in `CliqueLockHook.execute`

Status Acknowledged

Description The `execute` function in CliqueLockHook approves the `LOCK` contract for `_amount` tokens before creating a stream. However, for some ERC20 tokens such as USDT, setting a new allowance without first resetting it to zero will cause the transaction to revert. This is because USDT and similar tokens enforce that any non-zero allowance must first be set to zero before updating it to a new value. The current implementation does not account for this behavior, which may lead to failed executions when interacting with such tokens.

Impact Transactions using tokens like USDT may revert unexpectedly during repeated executions, causing legitimate stream creations to fail.

Recommended mitigation Before approving the `LOCK` contract, check the current allowance and reset it to zero if it is non-zero.

```
+   token.safeApprove(LOCK, 0);
_distributor().hookTransfer(address(this), _amount);
 token.safeApprove(LOCK, _amount);
 uint256 streamId = CliqueLock(LOCK).createStream(_streamConfig);
 streamIds[_root][_recipient] = streamId;
```

[L-04] Missing Validation in `CliqueLockHook.setStreamPreset`

Status Acknowledged

Description The `setStreamPreset` function allows the contract owner to set a `StreamPreset` struct for a given `configurationId`. Currently, there are no validation checks on the fields of the struct. For example, parameters such as pieceDuration, vestingDuration, or unlock percentages could be set to zero or values outside expected ranges, which may lead to incorrect vesting behavior or runtime errors when streams are created.

Impact Setting invalid values could potentially cause streams to revert on creation or failed during claims.

Recommended mitigation It is possible that the necessary validations are performed later during the corresponding functions; however, performing validation before setting the values is still strongly recommended, similar to CliqueLock.

```
+   error InvalidStream();
function setStreamPreset(bytes32 _configurationId, StreamPreset memory _streamPreset)
external onlyOwner {
// Validate durations
+    if(_streamPreset.pieceDuration == 0 && _streamPreset.vestingDuration == 0) revert
InvalidStream();
+    if (_streamPreset.startUnlockPercentage + _streamPreset.cliffUnlockPercentage >
1e18) revert InvalidSchedule();
 streamPresets[_configurationId] = _streamPreset;
}
```

[I-01] Unused constructor parameter in `TransferHook.sol`

Status Fixed

Description The TransferHook contract accepts `_projectAdmin` as a constructor parameter during deployment. However, this parameter is neither stored nor used within the contract, effectively making the constructor a no-op. While this does not introduce a direct security vulnerability, it can lead to confusion, and may signal incomplete or incorrect contract initialization logic.

Recommended mitigation Consider implementing the following changes:

```
-    constructor(address _projectAdmin) {}
+    constructor() {}
```

[I-02] Unused import

Status Fixed

Description `Distributor.sol` imports `IDistributor` but does not reference it anywhere. This import is redundant and can be safely removed to improve clarity and compilation efficiency.

```
-import {IDistributor} from "./IDistributor.sol";
```

[I-03] State Change Without Event

Description Some functions modify important state variables but do not emit corresponding events. This omission makes it difficult for off-chain services to track state changes reliably.

Found in `CliqueLockHook.sol`

```
function setStreamPreset(bytes32 _configurationId, StreamPreset memory _streamPreset)
external onlyOwner {
    function execute(address _vault, address _recipient, bytes32 _root, uint256 _allo-
catedamount, bytes memory _extra)
```

Found in `BaseDistributor.sol`

```
function setSigner(address _signer) external onlyRoles(PROJECT_ADMIN) {
    function setDynamicRecipient(bool _dynamicRecipient) external onlyRoles(PROJECT_ADMIN)
{
    function setIsWrappedToken(bool _isWrappedToken) external onlyRoles(PROJECT_ADMIN) {
```

[G-01] Unused Error

Status Fixed

Description Consider removing unused error to save gas

```
-    error TransferFailed(address token, address from, address to, uint256 amount);
-    error DefaultFeeNotSet();
-    error TransferFailed();
```

[G-02] Avoid zero assignment in `for` loops counters for gas efficiency

Status Acknowledged

Description: To optimize gas efficiency in a for loop, it is recommended to avoid explicitly setting the counter to zero, as it is already initialized to zero by default. Removing unnecessary variable assignments will further enhance the contract's gas efficiency.

```
-   for (uint256 i = 0; i < ...; ) {...}
+   for (uint256 i; i < ...; ) {...}
```

Safe Edges

REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US

SAFEEDGES.IN